

6节点Redis集群（3主3从）+ OpenResty部署方案

一、方案概述

本方案基于Docker Compose实现6节点Redis集群（3主3从）部署，满足Redis集群最小运行要求（至少3主3从），同时联动OpenResty（Nginx+Lua）服务，实现Redis集群与OpenResty的互通，支持配置挂载、数据持久化和自动重启，适用于开发、测试及小型生产环境。

核心组件：

- Redis集群：6个节点（3主3从），基于Redis 7.0轻量版（alpine镜像），开启集群模式和数据持久化
- OpenResty：基于alpine镜像，映射8080端口，挂载本地配置文件和脚本，依赖Redis集群启动
- 网络：专用桥接网络（redis-net），实现所有服务互通，隔离外部网络

二、完整Docker Compose配置

创建文件命名为 `docker-compose.yml`，复制以下完整配置，直接使用（无需修改核心配置，可根据需求调整端口和挂载路径）：

```
1  version: '3.8'
2
3  services:
4      # 3个Redis主节点（核心节点，负责数据存储和读写）
5      redis-master-1:
6          image: redis:7.0-alpine # 使用Redis 7.0轻量版镜像，体积小、启动快
7          container_name: redis-master-1 # 容器名称，便于操作和识别
8          command: redis-server --port 6379 --cluster-enabled yes --cluster-config-
9              file nodes.conf --cluster-node-timeout 5000 --appendonly yes
10             # 启动命令说明：
11             # --cluster-enabled yes: 开启Redis集群模式
12             # --cluster-config-file nodes.conf: 集群节点信息配置文件，存储节点地址、角色等信息
13             # --cluster-node-timeout 5000: 节点超时时间（5秒），超时未响应则判定为节点故障
14             # --appendonly yes: 开启AOF持久化，重启容器后数据不丢失
15             volumes:
16                 - ./redis_data/master1:/data # 数据卷挂载：本机当前目录redis_data/master1
17                 → 容器内/data目录
```

```
16     networks:
17         - redis-net # 加入专用网络, 实现与其他服务互通
18     restart: always # 容器意外退出后自动重启, 保证服务稳定性
19
20     redis-master-2:
21         image: redis:7.0-alpine
22         container_name: redis-master-2
23         command: redis-server --port 6379 --cluster-enabled yes --cluster-config-
file nodes.conf --cluster-node-timeout 5000 --appendonly yes
24         volumes:
25             - ./redis_data/master2:/data
26         networks:
27             - redis-net
28         restart: always
29
30     redis-master-3:
31         image: redis:7.0-alpine
32         container_name: redis-master-3
33         command: redis-server --port 6379 --cluster-enabled yes --cluster-config-
file nodes.conf --cluster-node-timeout 5000 --appendonly yes
34         volumes:
35             - ./redis_data/master3:/data
36         networks:
37             - redis-net
38         restart: always
39
40     # 3个Redis从节点 (备用节点, 主节点故障时自动切换)
41     redis-slave-1:
42         image: redis:7.0-alpine
43         container_name: redis-slave-1
44         command: redis-server --port 6379 --cluster-enabled yes --cluster-config-
file nodes.conf --cluster-node-timeout 5000 --appendonly yes
45         volumes:
46             - ./redis_data/slave1:/data
47         networks:
48             - redis-net
49         restart: always
50
51     redis-slave-2:
52         image: redis:7.0-alpine
53         container_name: redis-slave-2
54         command: redis-server --port 6379 --cluster-enabled yes --cluster-config-
file nodes.conf --cluster-node-timeout 5000 --appendonly yes
55         volumes:
56             - ./redis_data/slave2:/data
57         networks:
58             - redis-net
```

```
59     restart: always
60
61     redis-slave-3:
62         image: redis:7.0-alpine
63         container_name: redis-slave-3
64         command: redis-server --port 6379 --cluster-config-file nodes.conf --
cluster-node-timeout 5000 --appendonly yes
65         volumes:
66             - ./redis_data/slave3:/data
67         networks:
68             - redis-net
69         restart: always
70
71     # OpenResty服务 (Nginx+Lua, 可通过Lua脚本操作Redis集群)
72     openresty:
73         image: openresty/openresty:alpine # OpenResty轻量版镜像
74         container_name: openresty-redis # 容器名称
75         ports:
76             - "8080:80" # 端口映射: 本机8080端口 → 容器内80端口 (访问localhost:8080即可
进入OpenResty)
77         volumes:
78             # 挂载本地配置文件和脚本, 方便修改, 无需重启容器 (仅需重载Nginx)
79             - ./nginx.conf:/usr/local/openresty/nginx/conf/nginx.conf:ro
80             - ./lua:/usr/local/openresty/nginx/lua:ro
81             - ./lualib:/usr/local/openresty/nginx/lualib:ro
82             # :ro 表示只读权限, 防止容器误修改本地文件
83         networks:
84             - redis-net
85         depends_on:
86             # 依赖所有Redis节点, 确保Redis集群启动后再启动OpenResty
87             - redis-master-1
88             - redis-master-2
89             - redis-master-3
90             - redis-slave-1
91             - redis-slave-2
92             - redis-slave-3
93         restart: always
94
95     # 专用桥接网络: 所有服务在该网络内互通, 隔离外部网络, 提升安全性
96     networks:
97         redis-net:
98             driver: bridge
```

三、前置准备

在部署前，需在 `docker-compose.yml` 所在目录，创建以下文件和文件夹（确保挂载路径正确，避免启动失败）：

1. 创建 `nginx.conf` 文件：OpenResty的主配置文件（根据自身业务需求修改，基础配置可参考下方示例）；
2. 创建 `lua` 文件夹：存放Lua脚本（用于操作Redis集群、接口开发等）；
3. 创建 `lua-lib` 文件夹：存放Lua依赖库（如Redis操作相关的lua-resty-redis库）。

补充：nginx.conf基础示例（可直接复制使用，适配本部署方案）：

```
1  worker_processes  1;
2  error_log  /var/log/nginx/error.log warn;
3  pid       /var/run/nginx.pid;
4
5  events {
6      worker_connections  1024;
7  }
8
9  http {
10     include        /etc/nginx/mime.types;
11     default_type    application/octet-stream;
12
13     log_format  main  '$remote_addr [$time_local] "$request" '
14                      '$status $body_bytes_sent "$http_referer" '
15                      '"$http_user_agent" "$http_x_forwarded_for"';
16
17     access_log  /var/log/nginx/access.log  main;
18
19     sendfile    on;
20     keepalive_timeout  65;
21
22     server {
23         listen      80;
24         server_name localhost;
25
26         # 示例接口：通过Lua脚本访问Redis集群
27         location /redis-test {
28             default_type 'text/plain';
29             content_by_lua_block {
30                 local redis = require "resty.redis"
31                 local red = redis:new()
32
33                 -- 连接Redis集群（通过容器名访问，无需写IP）
34                 local ok, err = red:connect("redis-master-1", 6379)
35                 if not ok then
```

```

36         ngx.say("连接Redis失败: ", err)
37         return
38     end
39
40     -- 测试设置和获取值
41     red:set("test_key", "redis_cluster_test")
42     local res, err = red:get("test_key")
43     if res then
44         ngx.say("Redis返回值: ", res)
45     else
46         ngx.say("获取Redis值失败: ", err)
47     end
48
49     -- 关闭连接
50     red:close()
51 }
52 }
53
54 location / {
55     root    /usr/local/openresty/nginx/html;
56     index   index.html index.htm;
57 }
58
59 error_page 500 502 503 504 /50x.html;
60 location = /50x.html {
61     root    /usr/local/openresty/nginx/html;
62 }
63 }
64 }

```

四、部署步骤（一键执行，简单高效）

步骤1：启动所有容器

进入 `docker-compose.yml` 所在目录，执行以下命令，启动Redis集群和OpenResty服务（后台运行）：

```
1 docker-compose up -d
```

执行完成后，可通过以下命令查看容器运行状态（确保所有7个容器均为 `Up` 状态）：

```
1 docker-compose ps
```

步骤2：初始化Redis集群

启动容器后，Redis节点仅开启了集群模式，未形成真正的集群，需执行初始化命令，自动分配主从关系（3主3从）：

```
1 docker exec -it redis-master-1 redis-cli --cluster create \  
2 redis-master-1:6379 redis-master-2:6379 redis-master-3:6379 \  
3 redis-slave-1:6379 redis-slave-2:6379 redis-slave-3:6379 \  
4 --cluster-replicas 1
```

命令说明：

- `docker exec -it redis-master-1`：进入redis-master-1容器（任意一个Redis容器均可）；
- `redis-cli --cluster create`：创建Redis集群；
- 后面跟6个节点的地址（容器名:端口），容器名可直接解析为IP（因在同一网络）；
- `--cluster-replicas 1`：指定每个主节点对应1个从节点（3主→3从，符合最小集群要求）。

执行命令后，会提示集群分配方案，输入 `yes` 确认，等待1-2分钟，初始化完成。

步骤3：验证集群状态

初始化完成后，执行以下命令，验证Redis集群是否正常运行：

```
1 docker exec -it redis-master-1 redis-cli cluster info
```

若输出中包含 `cluster_state:ok`，则说明Redis集群部署成功，可正常使用；

也可查看集群节点信息，确认主从关系：

```
1 docker exec -it redis-master-1 redis-cli cluster nodes
```

步骤4：验证OpenResty与Redis集群连通性

OpenResty启动后，访问以下地址，验证是否能正常连接Redis集群：

```
1 curl localhost:8080/redis-test
```

若返回 Redis返回值: `redis_cluster_test`，则说明OpenResty与Redis集群连通正常。

五、核心说明与注意事项

5.1 数据持久化

每个Redis节点都挂载了独立的数据卷

(`./redis_data/master1~master3`、`./redis_data/slave1~slave3`)，开启了AOF持久化 (`appendonly yes`)，即使删除容器、重启服务器，数据也不会丢失；

若需清理数据，直接删除本地 `redis_data` 文件夹即可。

5.2 服务稳定性

所有服务均配置 `restart: always`，容器意外退出（如服务器重启、服务崩溃）后会自动重启，无需手动干预。

5.3 网络通信

所有服务均加入 `redis-net` 桥接网络，服务间可通过容器名直接通信（无需记住IP），例如OpenResty中可直接通过 `redis-master-1:6379` 连接Redis主节点。

5.4 常见问题

- 容器启动失败：检查本地 `nginx.conf` 文件是否存在、挂载路径是否正确，确保 `lua`、`lua-lib` 文件夹已创建；
- 集群初始化失败：检查所有Redis容器是否正常启动（`docker-compose ps`查看状态），若有容器未启动，执行 `docker-compose restart` 重启；
- OpenResty无法连接Redis：检查Redis集群是否初始化成功（`cluster_state:ok`），确认OpenResty容器已加入`redis-net`网络。

六、常用操作命令

1. 重启所有服务：`docker-compose restart`
2. 停止所有服务：`docker-compose down`（不删除数据卷，数据保留）
3. 停止并删除所有服务及数据卷：`docker-compose down -v`（谨慎使用，会删除所有Redis数据）
4. 查看Redis集群日志：`docker logs redis-master-1`（替换容器名可查看其他节点日志）

5. 进入Redis容器操作: `docker exec -it redis-master-1 sh`
6. 重载OpenResty配置（修改nginx.conf后无需重启容器）: `docker exec openresty-redis nginx -s reload`

（注：文档部分内容可能由 AI 生成）